



Figure 9: IBMTA deadlock detection

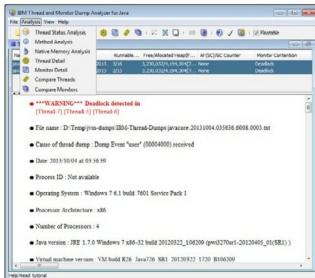


Figure 10: IBMTA analysis

time, the state changes each thread has undergone can be understood and, possibly, the reason behind a deadlocked or a poorly responding thread can be understood better. To compare thread dumps, load multiple thread dumps, choose the dumps to compare, right click and choose **Compare Threads**. Similarly, the monitors owned by threads in different dumps can be visualised by choosing **Compare Monitors**.

The native memory analysis gives information on the various memory areas, threads, JIT compiler details and class libraries in a single view. This is useful to analyse the overall memory profile of the JVM, i.e., analyse the memory utilisation of each of these memory areas. The Monitor Detail view is the most useful tool in quickly identifying the monitor dependencies amongst the threads that have led to a deadlock. The cyclic dependency amongst threads and the monitors that caused these dependencies can be visualised by drilling down the threads. Clicking on a thread gives information on the monitor owned by the thread and the monitor it is waiting on. It can be seen that this monitor is being held by another thread.

A variety of free tools is available to profile and analyse Java applications in order to find memory issues and performance bottlenecks. Most of the tools provide the basic information required to perform the analysis. Advanced features such as Object Query Language (OQL) can be used by experts to perform more detailed analysis. The choice of tool depends on the familiarity with the tool and the kind of JVM being monitored (IBM or Oracle). Expertise in the JVM memory and threading concepts is more important for

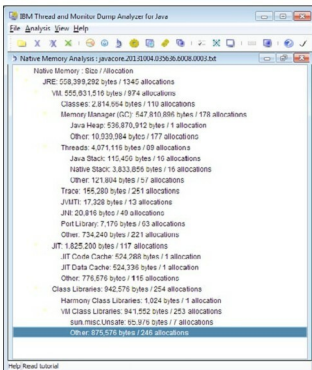


Figure 11: IBMTA thread status

analysing performance issues than for choosing a tool. 

References

- [1] Eclipse Memory Analysis Tool (MAT) www.eclipse.org/mat
- [2] Object Query Language - http://en.wikipedia.org/wiki/Object_Query_Language
- [3] IBM Heap Dumps http://publib.boulder.ibm.com/infocenter/ieduas/v1/r1/m0/index.jsp?topic=/com.ibm.ies.was_v6.0.1/PD/IASv601_zOS_Heapdumps/player.html
- [4] HPROF: A Heap/CPU Profiling Tool - <http://docs.oracle.com/javase/7/docs/technotes/samples/hprof.html>
- [5] Path to GC Queries: http://pic.dhe.ibm.com/infocenter/isa/v4r1/m0/index.jsp?topic=/%2Fcom.ibm.java.diagnostics.memory.analyzer.doc%2Fpath_to_gcroots.html
- [6] Java Tools: JPS: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jps.html>
- [7] Jstat: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jstat.html>
- [8] JMap: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jmap.html>
- [9] JHstat: <http://docs.oracle.com/javase/7/docs/technotes/tools/share/jhstat.html>
- [10] Java Instrumentation package: <http://docs.oracle.com/javase/7/docs/api/index.html#java.lang.instrument/package-summary.html>
- [11] Visual GC help: <http://www.oracle.com/technetwork/java/visualgc-736680.html>
- [12] IBM Developer Works Community: <https://www.ibm.com/developerworks/community/>
- [13] Java HPROF: <http://docs.oracle.com/javase/7/docs/technotes/samples/hprof.html>
- [14] Portable Heap Dump: http://pic.dhe.ibm.com/infocenter/java7sdk/v7.0/index.jsp?topic=/%2Fcom.ibm.java.v7.0.doc%2Fdiag%2Ftools%2Fheapdump_phd_format.html

By: Murali Suraparaju

The author holds an M Tech in Computer Science. He has worked extensively on building enterprise applications using JEE technologies, and has developed solutions for enterprise and embedded products. He is a member of the Performance Engineering practice in the Financial Services division of Infosys Limited.